

Jenkins Dashboard, Freestyle Jobs, Parameters & Environment Variables

Jenkins Dashboard and UI

What is Jenkins?

Jenkins is an **open-source automation server** used to implement **Continuous Integration (CI)** and **Continuous Delivery/Deployment (CD)**. It automates repetitive tasks such as building applications, running tests, and deploying software.

One of Jenkins' biggest strengths is its plugin ecosystem, which allows it to integrate with hundreds of different tools.

Key Features

- Open-source and free to use
 - Supports CI/CD pipelines
 - More than **2,000 plugins** available
 - Easy integration with Git, Docker, Kubernetes, Maven, SonarQube, Slack, AWS, and many more tools
 - Supports distributed builds using multiple agents
-

What is a Jenkins Job?

A **Jenkins Job** is a task or automation that Jenkins executes.

A job contains a sequence of steps that Jenkins performs automatically whenever the job is triggered.

Examples of Jenkins Jobs

- Build an application
- Run automated tests
- Execute shell scripts
- Package an application
- Deploy an application
- Send notifications after build completion

For example, in a Node.js application a Jenkins job may execute:

```
npm install
```

```
npm start
```

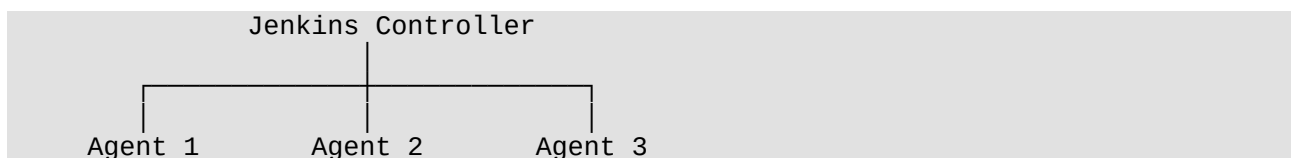
or

```
npm install
npm test
npm run build
```

Jenkins Controller and Agents

Think of Jenkins like a **manager** in a company.

- The **Controller (Master)** receives work.
- It assigns jobs to available **Agents (Workers)**.
- Agents execute the tasks and return the results back to the controller.



The Jenkins Controller can run on:

- Linux
- Windows
- macOS

Jenkins itself is a **Java application**, so Java must be installed before installing Jenkins.

Lab Only: Disable Jenkins Authentication (Not Recommended)

Warning

This method should only be used in a **practice or lab environment**. Never disable Jenkins authentication on a production server.

Step 1

Stop the Jenkins service.

```
sudo systemctl stop jenkins.service
```

Step 2

Open the Jenkins configuration file.

```
sudo nano /var/lib/jenkins/config.xml
```

Step 3

Locate:

```
<useSecurity>true</useSecurity>
```

Change it to:

```
<useSecurity>false</useSecurity>
```

Step 4

Save the file.

Step 5

Start Jenkins again.

```
sudo systemctl start jenkins.service
```

After restarting, Jenkins will no longer ask for user authentication.

Jenkins Labs

LAB 1 – Installing Jenkins on Ubuntu Linux

Refer to the GitHub repository for the installation steps.

Repository:

<https://github.com/ajit010/Jenkins-6th-july.git>

Complete the installation before moving to the next lab.

LAB 2 – Initial Setup and Unlocking Jenkins

Complete the initial Jenkins setup by following the instructions provided in the GitHub repository.

Tasks include:

- Unlock Jenkins
 - Install suggested plugins
 - Create the first administrator user
 - Log in to the Jenkins Dashboard
-

LAB 3 – Exploring the Jenkins Dashboard and UI

Spend some time understanding the Jenkins interface.

Important sections include:

Jobs

Jenkins supports different job types, including:

- Freestyle Project
 - Pipeline
 - Multibranch Pipeline
 - Folder
 - Organization Folder
-

Build History

Displays all previous builds.

Example:

#1
#2
#3

You can click any build to view:

- Build status
 - Console Output
 - Build duration
 - Build artifacts
 - Build logs
-

Manage Jenkins

This section contains all administration options.

Common settings include:

- System Configuration
- Global Tool Configuration
- Environment Variables
- Nodes

- Credentials
- Plugins
- Security
- Cloud Configuration

Spend a few minutes exploring each option.

LAB 4 – Creating Your First Freestyle Job

Step 1

Click **New Item** (or **Create a New Job**).

Step 2

Enter the job name.

Example:

```
hello-world-job
```

Select:

```
Freestyle Project
```

Click **OK**.

Step 3

Under **Description**, enter:

```
My first Jenkins job
```

Step 4

Scroll to **Build Steps**

Click

```
Add Build Step
```

Choose

```
Execute Shell
```

Paste:

```
echo "Hello World from Jenkins"
echo "Today is $(date)"
echo "Running on host: $(hostname)"
```

Step 5

Click **Save**.

Step 6

Click

Build Now

The build will appear in the Build History section.

Open the build and verify the output.

LAB 5 – Reading Console Output and Build Status

After a build completes, Jenkins displays its status.

Successful Build

Displayed with a **blue/green icon** depending on the Jenkins theme.

Failed Build

Displayed with a **red icon**.

Open the build.

Click:

Console Output

Review every step executed by Jenkins.

The console log contains:

- Commands executed
 - Output generated
 - Errors
 - Exit status
 - Build duration
-

Troubleshooting Tip

If a build fails:

1. Download the console log.
2. Upload it to ChatGPT or Claude.
3. Use a prompt like:

```
Analyze this Jenkins build log.
```

```
Identify the root cause of the failure and provide step-by-step troubleshooting instructions with possible fixes.
```

This is an effective way to learn how to debug Jenkins jobs.

LAB 6 – Multiple Build Steps in One Job

A Jenkins job can contain multiple build steps.

Create three separate **Execute Shell** build steps.

Shell Step 1

```
echo "Step 1: Preparing Environment"
mkdir -p /tmp/jenkins-demo
```

Shell Step 2

```
echo "Step 2: Creating a file"
echo "Build data: $(date)" > /tmp/jenkins-demo/output.txt
```

Shell Step 3

```
echo "Step 3: Displaying the file"
cat /tmp/jenkins-demo/output.txt
```

Build the job.

Observe how Jenkins executes each build step sequentially.

LAB 7 – Parameterized Builds

Parameterized builds allow users to provide input values before starting a build.

Instead of creating multiple jobs for different environments or users, a single job can accept parameters.

Step 1

Create a new Freestyle Project.

Name it:

parameterized-job

Step 2

Under **General**

Enable:

This project is parameterized

Step 3

Add a **String Parameter**

Name:

USERNAME

Default Value:

student

Description:

Your name

Step 4

Add a **Choice Parameter**

Name:

ENVIRONMENT

Choices:

dev

```
staging
prod
```

Step 5

Add a **Boolean Parameter**

Name:

```
VERBOSE
```

Leave it unchecked by default.

Step 6

Add an **Execute Shell** build step.

```
echo "Hello, $USERNAME"

echo "Deploy target: $ENVIRONMENT"

if [ "$VERBOSE" = "true" ]; then
    echo "Verbose mode enabled"
    env | sort | head -20
fi
```

Step 7

Save the job.

Notice the left menu now shows:

```
Build with Parameters
```

Click it.

Try different combinations of:

- USERNAME
- ENVIRONMENT
- VERBOSE

Observe how the build output changes based on the selected values.

Real-World Example

Imagine the same application needs to be deployed to different environments.

Example infrastructure:

Environment	Server Configuration
Development	4 vCPU, 16 GB RAM
Production	16 vCPU, 64 GB RAM

Instead of creating multiple Jenkins jobs, one parameterized job can deploy to different environments by selecting:

- dev
- staging
- prod

This approach makes Jenkins jobs more reusable and easier to maintain.

LAB 8 – Working with Jenkins Environment Variables

Jenkins automatically provides several built-in environment variables during every build.

These variables provide useful information about the current job and build.

Step 1

Create a Freestyle Project named:

```
env-explorer
```

Step 2

Add an **Execute Shell** build step.

```
echo "JOB_NAME = $JOB_NAME"
echo "BUILD_NUMBER = $BUILD_NUMBER"
echo "BUILD_ID = $BUILD_ID"
echo "BUILD_URL = $BUILD_URL"
echo "WORKSPACE = $WORKSPACE"
echo "JENKINS_URL = $JENKINS_URL"
echo "NODE_NAME = $NODE_NAME"
```

Step 3

Add another Execute Shell step.

```
env | sort
```

This prints all available environment variables.

Step 4

Save the job.

Click **Build Now**.

Study the Console Output.

Step 5

Open the following URL in your browser:

```
http://<server-ip>:8080/env-vars.html
```

This page lists all Jenkins environment variables available during a build.

Step 6

Run the job twice.

Observe:

- BUILD_NUMBER increases with every build.
 - JOB_NAME remains the same.
-

Linux Environment Variables

You can also create your own environment variables in Linux.

Create a variable:

```
export DB_USERNAME=ajit_db
```

Verify it:

```
echo $DB_USERNAME
```

If the value is displayed, the environment variable has been successfully created.

Note: Variables created using `export` are temporary and exist only for the current shell session unless added to shell configuration files such as `.bashrc` or `.profile`.

LAB 9 – Scheduling Builds Using Cron

Jenkins can automatically trigger jobs on a schedule using **Cron expressions**. This is useful for running builds, backups, health checks, or automated tasks without manual intervention.

Example Use Cases

- Nightly builds
 - Daily backups
 - Running automated tests every few hours
 - Health check scripts
 - Scheduled deployments
-

Understanding Cron Syntax

A cron expression consists of **five fields**:

MINUTE	HOUR	DAY_OF_MONTH	MONTH	DAY_OF_WEEK
--------	------	--------------	-------	-------------

Example:

```
* * * * *
```

This means **run every minute**.

Step 1

Create a new **Freestyle Project** named:

```
scheduled-job
```

Step 2

Add an **Execute Shell** build step.

```
echo "Scheduled run at $(date)"
```

Step 3

Scroll to **Build Triggers**.

Enable:

Build periodically

Enter the following schedule:

H/2 * * * * *

This runs the job approximately every **2 minutes**.

What does H mean?

Unlike standard Linux cron, Jenkins supports the **H (Hash)** symbol.

Instead of triggering every job at the exact same time, Jenkins distributes jobs across different minutes to reduce server load.

For example:

H/2 * * * * *

means:

Run every 2 minutes, but choose the starting minute automatically.

Step 4

Save the job.

Wait for **5–10 minutes**.

Jenkins should automatically trigger the build.

The build history will show:

Started by timer

More Cron Examples

Every day at 2:30 AM

30 2 * * *

Every weekday at 9:00 AM

0 9 * * 1-5

Twice a day (Load Balanced)

```
H H(0-7)/12 * * *
```

Stop Scheduled Builds

To stop automatic execution:

- Remove the cron expression, or
- Comment it out

Then save the job.

LAB 10 – Installing and Managing Plugins

One of Jenkins' biggest strengths is its **plugin ecosystem**.

Plugins extend Jenkins by adding support for additional tools and technologies.

Examples include:

- Docker
 - Kubernetes
 - Git
 - Maven
 - SonarQube
 - Slack
 - AWS
 - Build Timestamp
 - Docker Pipeline
-

Step 1

Navigate to:

Manage Jenkins

↓

Plugins

Step 2

Search for and install plugins such as:

- Git
- Docker
- Docker Pipeline
- Build Timestamp

Tip

Install only the plugins you actually need. Too many unused plugins increase maintenance effort and may introduce compatibility issues.

Step 3

Restart Jenkins after installation.

```
sudo systemctl restart jenkins
```

Wait for **1–2 minutes**, then open:

```
http://<Public-IP>:8080
```

Sign in using your Jenkins credentials.

LAB 11 – Cloning a Git Repository Using a Freestyle Job

In this lab, Jenkins will clone source code directly from a GitHub repository.

We will use the official Jenkins sample project:

Repository:

```
https://github.com/jenkins-docs/simple-java-maven-app.git
```

Prerequisites

- Jenkins installed and running
- Git installed on the Jenkins server
- Git Plugin installed in Jenkins

Verify Git installation:

```
git --version
```

Step 1

Create a Freestyle Project.

Name it:

```
git-clone-demo
```

Step 2

Under:

```
Source Code Management
```

Select:

```
Git
```

Step 3

Repository URL

```
https://github.com/jenkins-docs/simple-java-maven-app.git
```

Step 4

Since the repository is public:

- Leave **Credentials** empty.
- Set **Branch Specifier** to:

```
*/master
```

Step 5

Add an **Execute Shell** build step.

```
pwd
```

```
ls -la
```

```
git log --oneline -5
```

This verifies that Jenkins successfully cloned the repository.

Step 6

Click:

Build Now

Watch the Console Output.

You will notice Jenkins first performs the Git clone operation before executing the shell commands.

Step 7

Open:

Workspace

from the left menu.

Browse the cloned project files directly from the Jenkins UI.

LAB 12 – Polling SCM for Changes

Instead of scheduling builds at fixed times, Jenkins can periodically check a Git repository and trigger a build **only when new commits are detected**.

This feature is called **Poll SCM**.

Step 1

Fork the repository into your own GitHub account.

<https://github.com/jenkins-docs/simple-java-maven-app.git>

Step 2

Use the Jenkins job created in **Lab 11**.

Update the repository URL so that it points to **your fork**.

Step 3

Go to:

Build Triggers

Enable:

```
Poll SCM
```

Use the schedule:

```
H/2 * * * * *
```

Jenkins will check GitHub approximately every two minutes.

Step 4

Click:

```
Git Polling Log
```

Initially you should see:

```
No changes
```

Step 5

Open GitHub.

Edit the **README.md** file.

Commit the change directly from GitHub.

Step 6

Wait a few minutes.

Jenkins detects the new commit.

A new build starts automatically.

Build Cause:

```
Started by an SCM change
```

This means Jenkins detected changes in the repository and triggered the build automatically.

LAB 13 – Installing Maven and Building a Java Project

In this lab, Jenkins will build a real Java application using **Apache Maven**.

Install Maven on the Jenkins Server

```
sudo apt update
```

```
sudo apt install -y maven
```

Verify installation:

```
mvn --version
```

Alternative Method (Recommended)

Instead of installing Maven manually, Jenkins can install and manage Maven automatically.

Navigate to:

Manage Jenkins

↓

Tools

↓

Maven Installations

Click:

Add Maven

Example name:

maven-3.9

Enable:

Install automatically

Jenkins downloads Maven whenever required.

Repository

Use the following GitHub repository:

```
https://github.com/ajit010/devops-maven-docker.git
```

Configure the Job

Update the repository URL to the Maven project.

Build Step

Add an **Execute Shell** build step.

```
mvn clean  
mvn package
```

or

```
mvn clean package
```

Build the Project

Click:

```
Build Now
```

If the build succeeds, Maven creates the project artifact.

Verify the Output

Open the workspace directory:

```
/var/lib/jenkins/workspace/<your-job-name>
```

Navigate to:

```
target/
```

You should see the generated application artifact such as:

```
.war
```

or

```
.jar
```

depending on the project packaging type.

Docker vs Kubernetes

Understanding the difference between Docker and Kubernetes is important for modern DevOps workflows.

Docker

Docker runs containers on a **single host machine**.

Example:

```
EC2 Instance
|
Docker Engine
|
Containers
```

If the EC2 instance is stopped:

- All containers stop.
- When the server starts again, containers remain stopped.
- Containers must be started manually.

Example:

```
docker start <container-name>
```

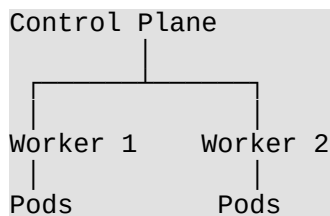
Docker by itself does **not** provide automatic container recovery after a host restart.

Kubernetes

Kubernetes manages containers across **multiple machines (nodes)**.

A Kubernetes cluster typically consists of:

- Control Plane (Master Node)
- Worker Nodes



If one worker node goes down:

- Kubernetes detects the failure.
 - The affected Pods are automatically recreated.
 - Workloads are scheduled on healthy worker nodes.
 - Applications remain available with minimal downtime.
-

Why Kubernetes?

Kubernetes provides features such as:

- High Availability (HA)
- Automatic Self-Healing

- Auto Scheduling
- Scaling
- Rolling Updates
- Load Balancing

Unlike Docker alone, Kubernetes continuously monitors the desired state of applications and works to maintain it automatically.

Summary

In this session, you learned how to:

- Schedule Jenkins jobs using Cron expressions
 - Understand the purpose of the H symbol in Jenkins Cron
 - Install and manage Jenkins plugins
 - Clone Git repositories using Jenkins
 - Automatically trigger builds using Poll SCM
 - Install and configure Maven
 - Build a Java project using Maven
 - Locate generated build artifacts (.jar / .war)
 - Understand the key differences between Docker and Kubernetes
 - Learn why Kubernetes provides high availability and self-healing compared to Docker alone
-